

CODE

```
x = 5 + 3 * 2  
print(x)
```

¿Cómo convertimos este texto en el resultado 11?

Descubriendo la magia detrás de nuestro intérprete.

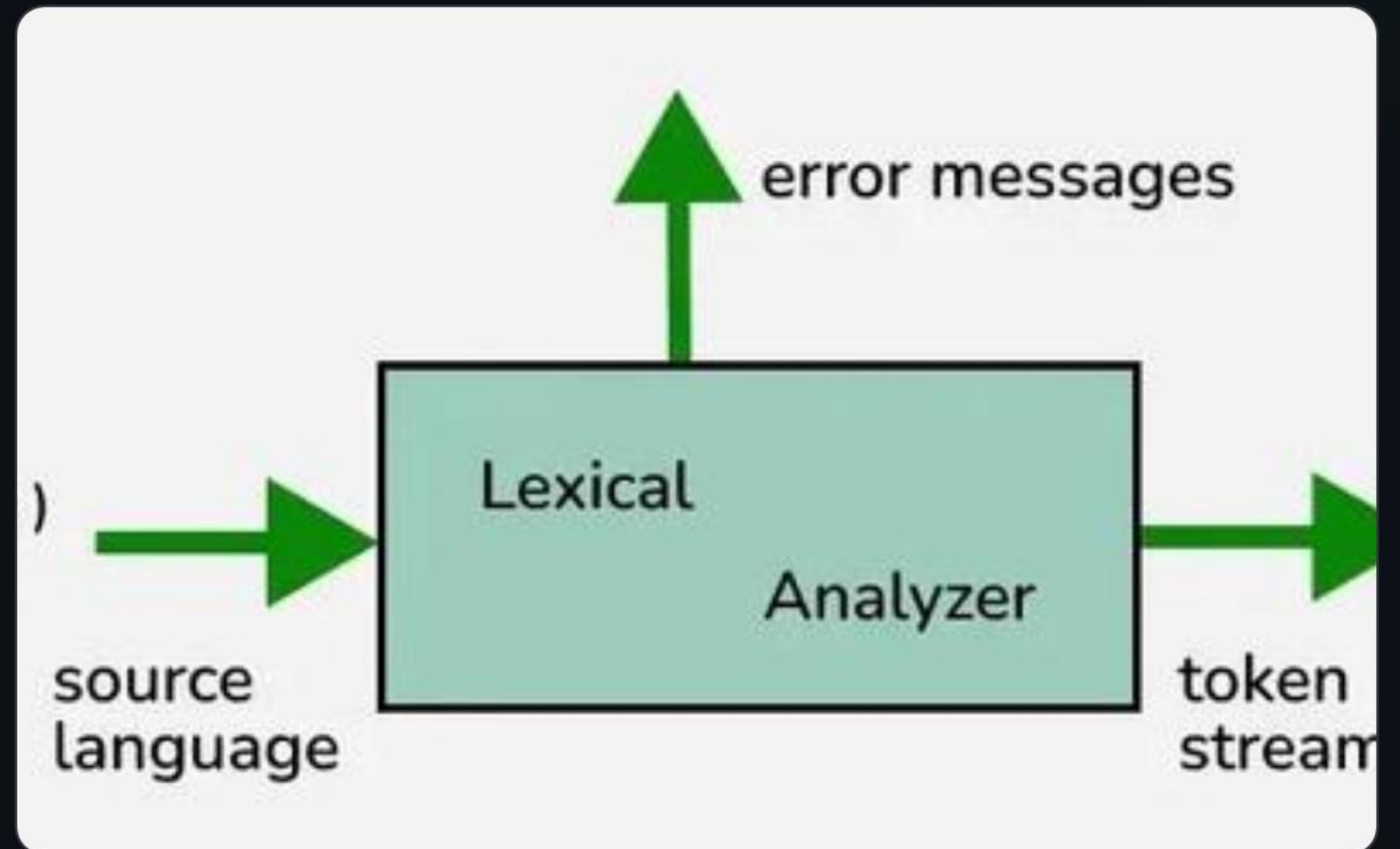
PASO 1: EL LEXER RECIBE EL TEXTO

```
Token Lexer::getNextToken()
```

CODE

- 🔧 **Lee:** Analiza carácter por carácter.
- 🔍 **Reconoce:** Identifica patrones lógicos.
- 🏷️ **Genera:** Produce una secuencia de Tokens.

"Convertimos texto bruto en unidades con sentido atómico."



DEBUG DEL LEXER

Carácter leído	Acción / Token Generado
x	IDENTIFIER(x)
=	EQUAL
5	NUMBER(5)
+	PLUS
3	NUMBER(3)
*	MUL
2	NUMBER(2)

PASO 2: EL PARSER ENTRA EN ACCIÓN

```
expr()
  └── term()
      └── factor()
```

CODE



Gramática BNF

El parser sigue reglas estrictas. Gracias a esta jerarquía, la multiplicación adquiere prioridad sobre la suma automáticamente.

```
// Estructura para: 5 + 3 * 2
expr() → term(factor(5)) + term(factor(3) * factor(2))
```

CODE

CONSTRUCCIÓN DEL AST

1. Inicio

5

Primer token detectado como nodo raíz inicial.

2. Suma

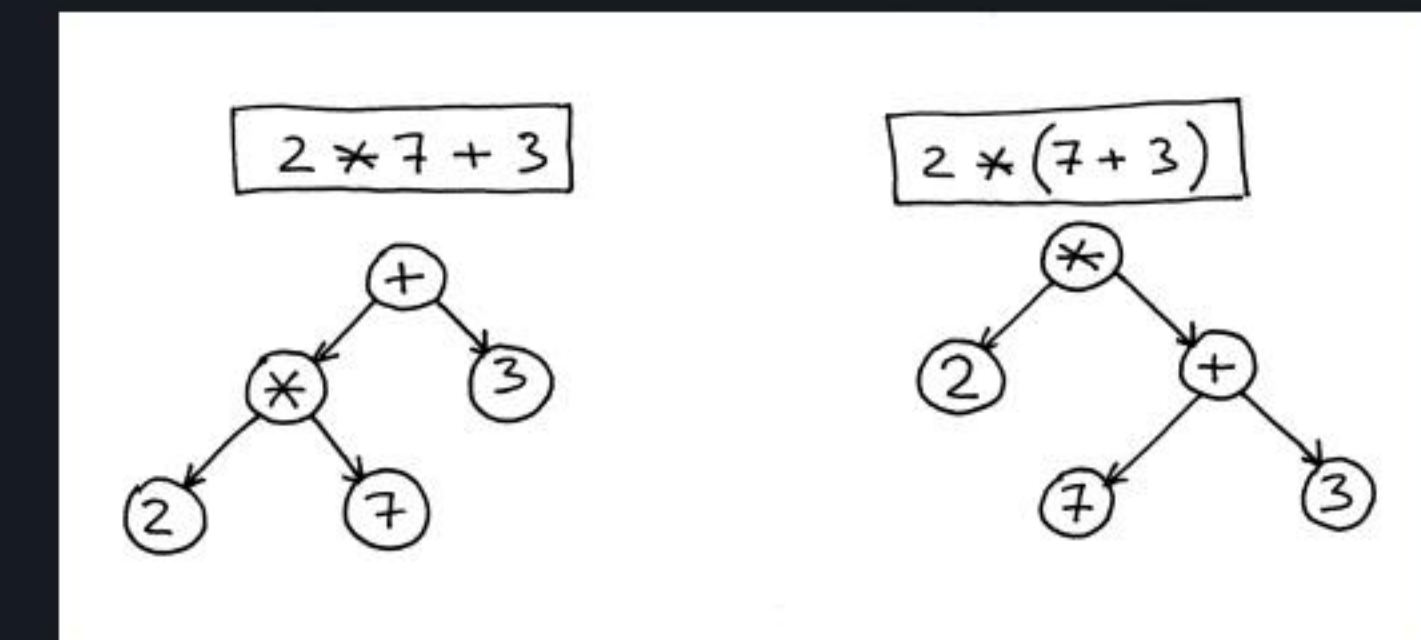
+

5

3

Se reconoce el operador binario.

3. Final



El árbol crece hacia abajo respetando la precedencia.

STACK DE LLAMADAS DEL PARSER

- > **parse()** : Inicio del proceso.
- > **statement()** : Reconoce $x = \dots$
- > **expression()** : Analiza la operación.
- > **term() / factor()** : Llega a los números.

CODE

```
factor() → return 5;  
factor() → return 3;  
factor() → return 2;
```

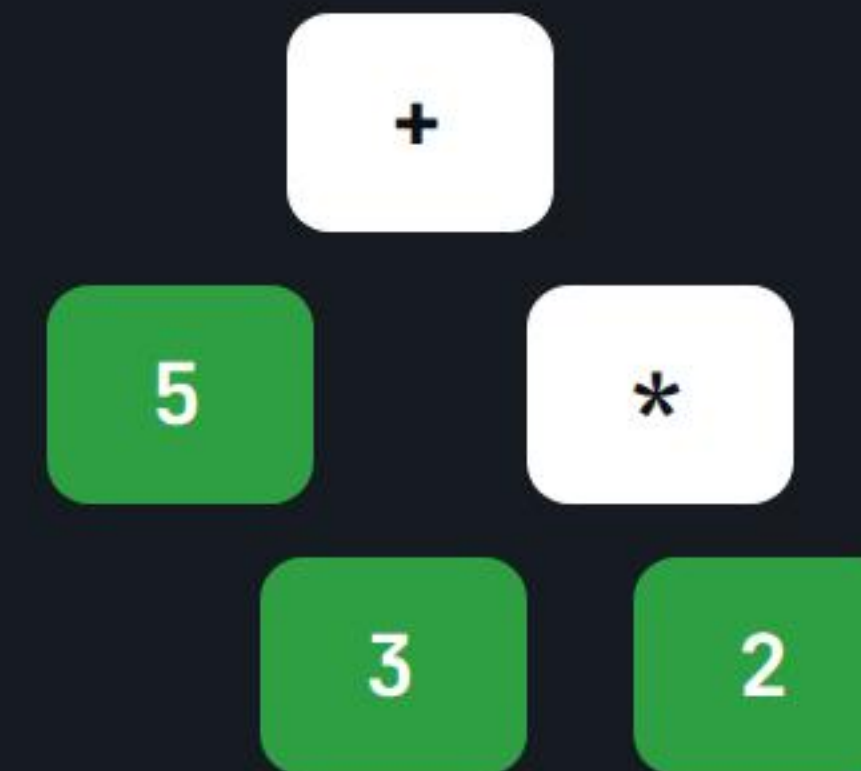
Estas funciones se invocan recursivamente para construir la estructura de datos.

PASO 3: EVALUACIÓN (TREE WALK)

↑ **Bottom-Up:** Se recorre de abajo hacia arriba.

🧮 **eval(*):** $3 * 2 = 6$

+ **eval(+):** $5 + 6 = 11$



ALMACENAMIENTO: X = 11

Nodo de Asignación

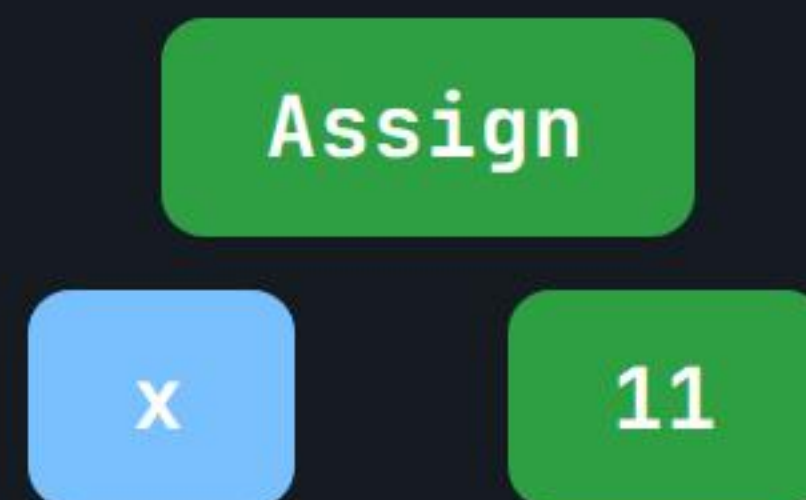


Tabla de Símbolos

Variable	Valor
x	11

EJECUCIÓN: PRINT(X)

- 🔍 buscar('x') en Tabla.
- ✓ Encontrado: 11.
- Enviar al Stream de salida.

OUTPUT

11

ARQUITECTURA DEL PROCESO



```
x = 5 + 3 * 2  
print(x)
```

CODE



> 11

CODE